#: Python program Flow Control/Conditional blocks:-

A Program's Control flow is the Order in which the program's Code executes.

The Control flow of a Python program is regulated by conditional statements, loops, and function calls.

- Python has 3 types of control Structures:-
  1. Sequential — default Mode.
  2. Selection — used for decisions and branching.
  3. Repetition — used for looping i.e repeating a piece of code multiple times.

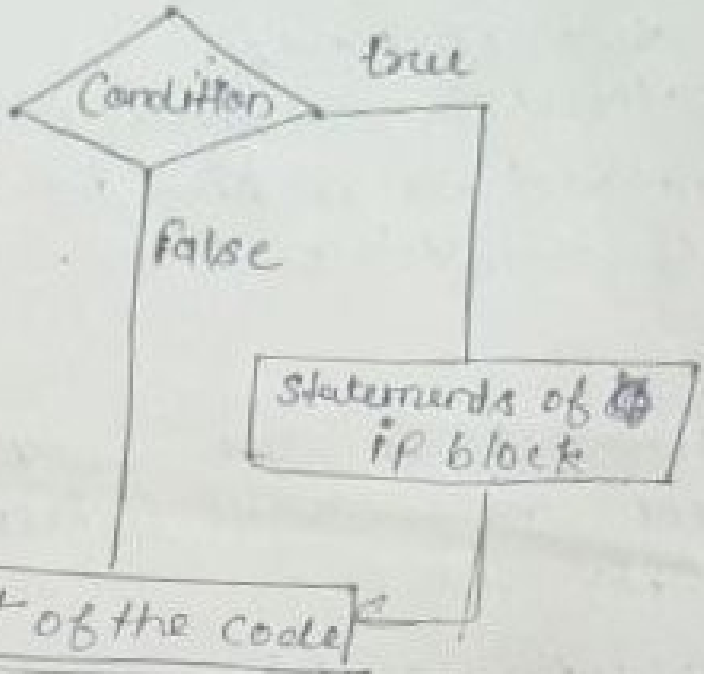
If:-

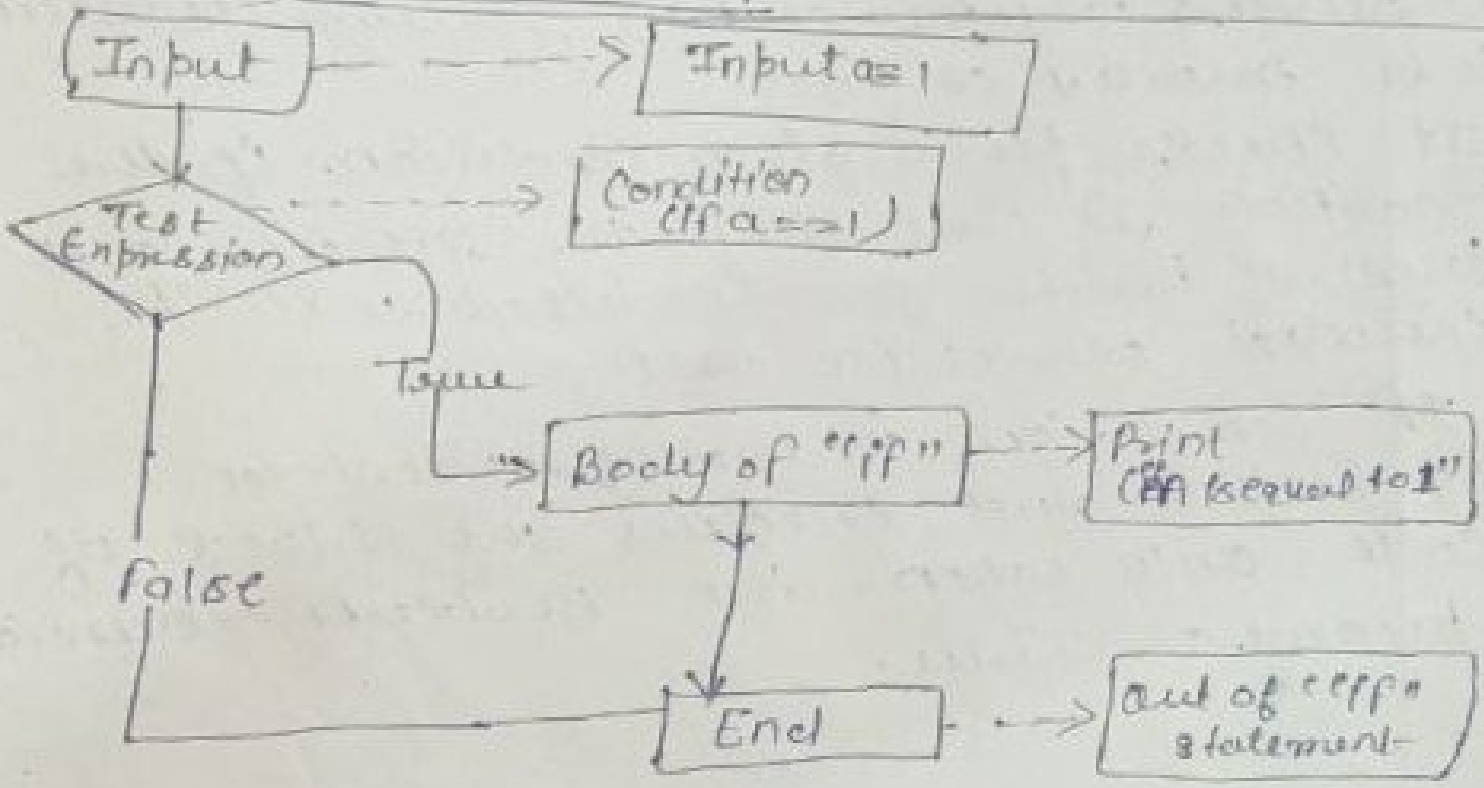
- Python if statement is one of the most commonly used conditional statements in Programming language.
- It decided whether certain statements need to be executed or not.
- It checks for a given condition, if the condition is true, then the set of code present inside the if block will be executed otherwise not.
- If condition evaluates a Boolean expression and executes the block of code only when the Boolean expression becomes True.

Syntax :-

```
If (EXPRESSION == TRUE):
    Block of code
else:
    Block of code.
```

- The condition will be evaluated to true or false.





ple :-

```
num = 5
If (num < 10):
    Print ("Num is smaller than 10")
Print ("This statement will always be executed")
```

Output :- Num is smaller than 10.
This statement will always be executed.

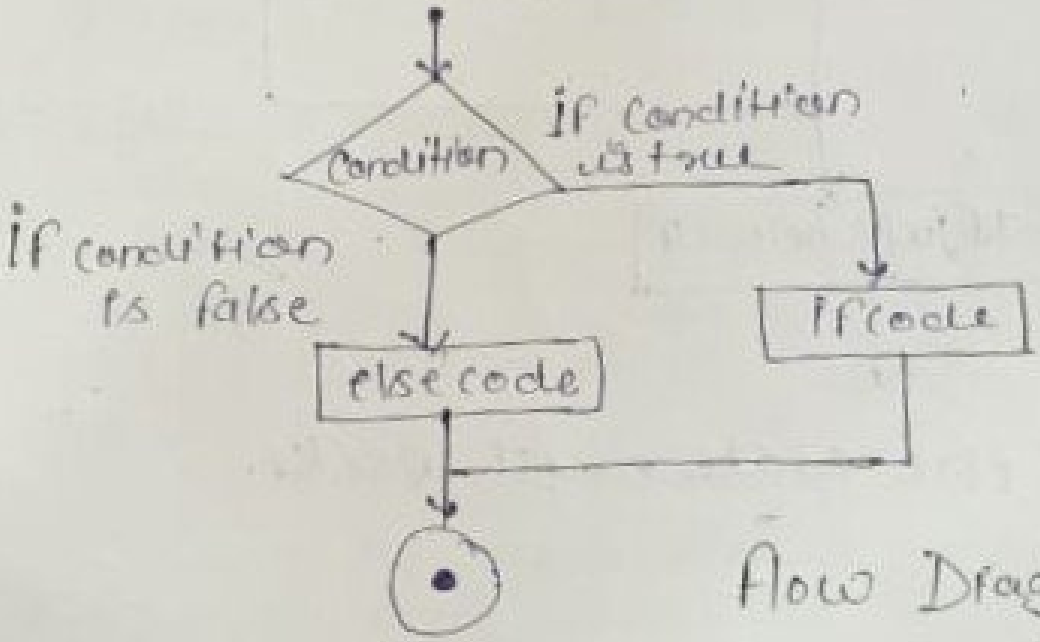
else and else if :-

An else statement can be combined with an if statement. An else statement contains block of code that executes if the conditional expression in the if statement resolves to 0 or a FALSE value.

- The else statement is an optional statement and there could be at most only one else statement following if.

Syntax :-

```
If expression:
    statement(s)
else:
    statements(s)
```



Flow Diagram

If else :

The If-else statement is used to execute both the true part and the false part of a given condition.

- If condition is true, the If block code is executed and if the condition is false, else block code is executed.

Syntax:-

```
If (condition):
# Executes this block if the condition is true
else:
# Executes this block if the condition is false.
```

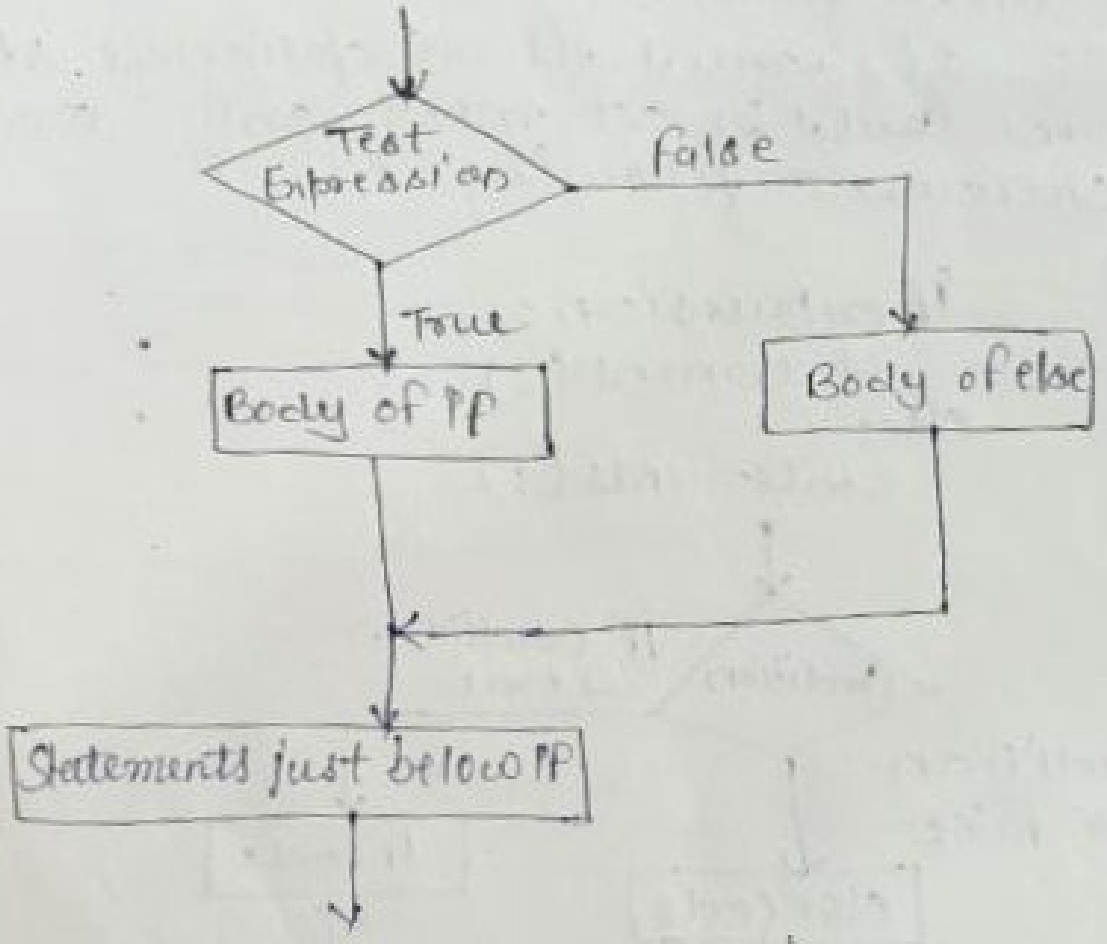


Fig:- If else statement works.

Simple for loops in Python :-

r loop :-

A for loop is used for iterating over a sequence
(list, tuple, dictionary, set or string.
→ This is less like for keyword in other programming languages, and works more as Iterator method as found in other object-oriented programming language.
→ For loop we can execute a set of statements once for each item in a list, tuple, set etc.
→ For loops are used for sequential traversal.
→ For loops can iterate over any of these iterable objects.

Syntax :-

```
for iterator_var in sequence
    statements(s)
```

For Example :-

```
fruits = ["apple", "orange", "kiwi"]
# Creating an iterator object
# from that iterable i.e fruits
iter_obj = iter(fruits)

# Infinite while loop
while True:
    try:
        # getting the next item
        fruit = next(iter_obj)
        print(fruit)
    except StopIteration:
        # If StopIteration is raised,
        # break from loop
        break
```

Output :-
apple
orange
kiwi

For loop using ranges :-

- In Programming, you often want to execute a block of code multiple times. So use a for loop.
- With the help of the range() function, we may produce a series of numbers.
- We can specify start, stop and size values in the manner range.
- If the step size is not specified, it defaults to 1.

Syntax :-

```
For index in range(n):
    Statement
```

* In this syntax, the index is called a loop counter. And n number of times start the loop will execute the statement
* The name of the loop counter doesn't have to be index. You can use whenever name you want
* The range() is a built-in function.
* The range(n) generates a sequence of n integers starting at zero. It increase the value by one until it reaches n.
* The range(n) generates a sequence of numbers: 0, 1, 2, ---. n-1.

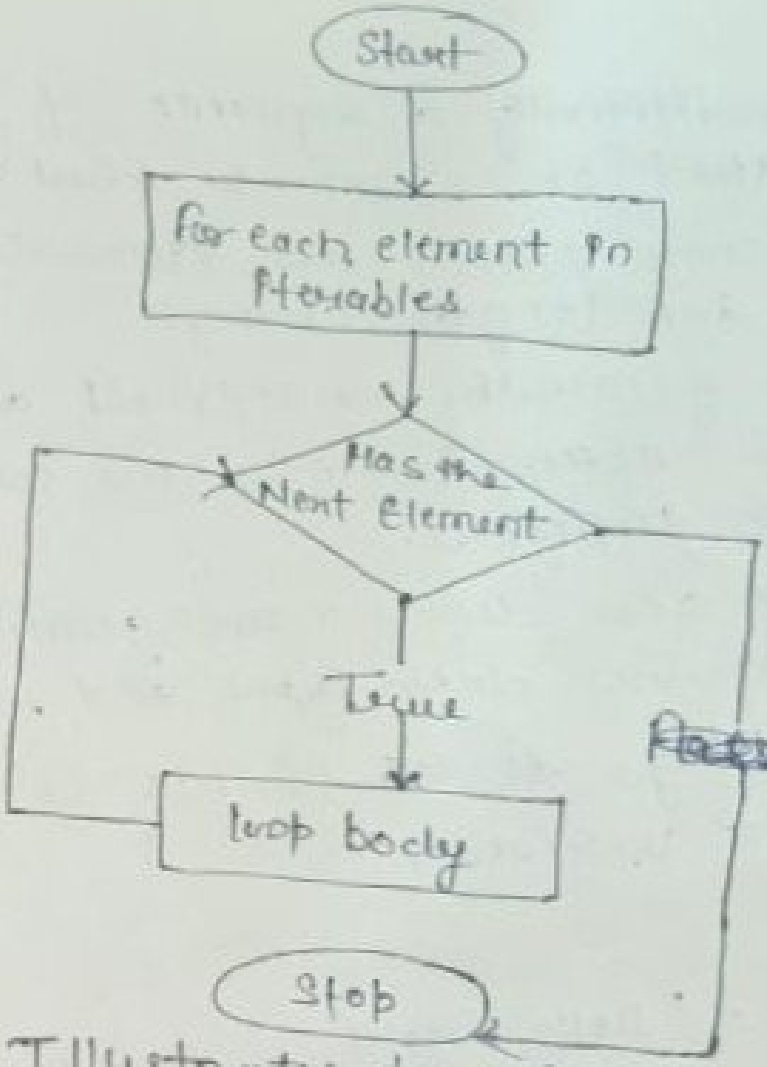


Diagram Illustrates the for loop Statement

Strings:-

- A string is traditionally a sequence of characters, either as a literal constant or as some kind of variable. The latter may allow its elements to be mutated and the length changed.
- A string is generally considered as a data type and is often implemented as an array data structure.
- String may also denote more general arrays or other sequence data types and structures.
- Creating strings is as simple as assigning a value to a variable.

Example:-

```
Var1 = 'Hello World'
Var2 = "Python programming"
```

Accessing Values in String:-

Python does not support a character type; these are treated as string of length one, thus also considered a substring

```
Var1 = 'Hello World!'
Var2 = "Python Programming"
Print "Var1[0]:", Var1[0]
Print "Var2[1:5]:", Var2[1:5]
```

Output

```
Var1[0]: H
Var2[1:5]: ytho
```

Updating Strings:-

Update an existing string by (re) assigning a variable to another string.

The new value can be ~~create~~ related to its previous value or to a completely different string altogether.

List :-

Python list is an ordered and changeable collection of data objects. An array which can contain objects of a single type, a list can contain a mixture of objects.

- A Python list of 5 objects.
- A list can contain objects of any data type including another list.
- A list is a sequence data type which is used to store the collection of data.
- Tuple and String are other types of sequence data types.

Example:-

```
Var1 = ["Geeks", "for", "Geeks"]
Print (Var1)
```

Output:-

["Geeks", "for", "Geeks"].

Example:-

```
# Creating a list with
# the use of Numbers
# (Having duplicate values)
List = [1,2,4,4,3,3,3,6,5]
Print ("\nList with the use of Numbers:")
Print (list)

# Creating a list with
# mixed type of values
# (Having numbers and string)
List= [1,2 'Geeks',4,'for',6, 'Geeks']
Print ("\nList with the use of mixed values:")
Print (List)
```

Output :-

List with the use of Numbers :
[1, 2, 4, 4, 3, 3, 3, 6, 5]
List with the use of mixed values:
[1, 2, 'Geeks', 4, 'For', 6, 'Geeks']

[redacted] :-

Dictionaries :-

- A Dictionary is kind of data structure that stores items in key-value pairs.
- A key is a unique identifier for an item and a value is the data associated with that key.
- ~~Dictionaries~~
- Dictionaries often store information such as words and definitions, but they can be used for much more.
- ~~Dictionaries~~ Dictionaries are mutable in python which means they can be changed after they are created.
- They are also unordered, indicating the items in a dictionary are not stored in any particular order.
- Creating a Dictionary.
- Complexities for Creating Dictionary
- Changing and Adding Elements to a Dictionary
- Accessing Elements of a Dictionary.

Use of While loops In Python :-

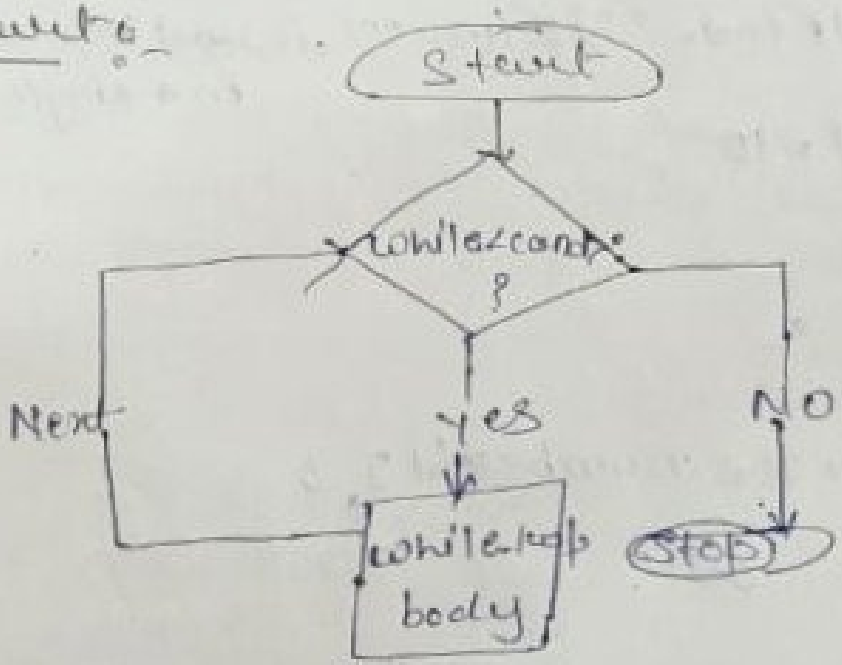
- Python while loop is used to execute a block of statements repeatedly until a given condition is satisfied.
- While loop statement is used to repeat a statement or group of statements until a given condition return false.
- When we are not certain about the number of times of loop requires execution, we will make use of while loop.

Syntax :-

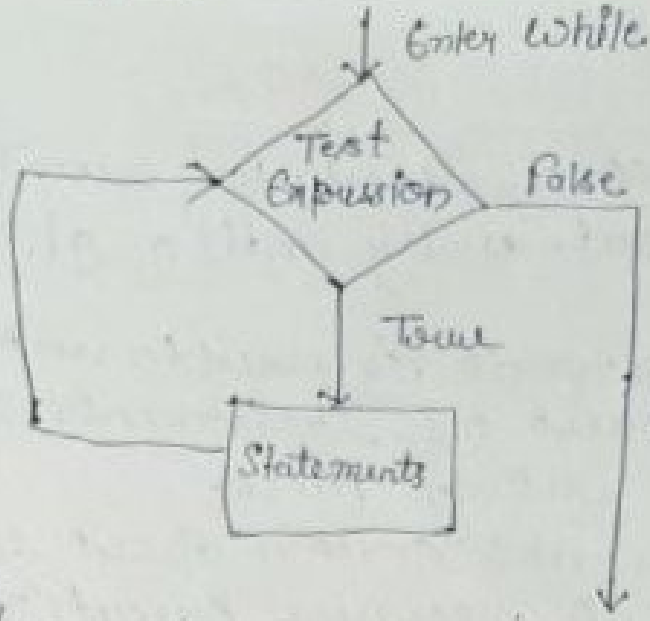
~~Condition:~~
~~Statement(s) # Body of loop statement~~

Condition:
Statement(s): # Body of loop that has set of statements
which requires repeated execution

Flow chart :-



Working of loop



flow chart of while loop

Example:-

```
# Program for the demonstration of while loop.
# Program to print the reverse of a given number
# loop will repeat itself until n!=0
n = int (input ("Enter the number: "))      # variable to store the number
while n != 0:
    d = n % 10
print (d, end = " ")  # (end="") is used to print the output in a single number
n //= 10
```

Output :-

```
Enter the number: 123
321
```

Loop manipulation using pass :-

Loops in Python automates and repeats the task in an efficient manner. But there may arise a condition where you want to exit the loop completely, skip an iteration or ignore that condition.

These can be done by loop control statements. Loop control statements changes execution from their normal sequence.

- Python supports the following control statements:-

1. Break statement
2. Continue statement
3. Pass statement

* Break Statement :-

The break statement in Python is used to terminate the loops or statement that is present. The control will pass to the statements that are present after the break statement, if available.

If the break statement is present in the nested loop, then it terminates only those loops which contain the break statement.

Syntax :-

```
for / while loop :
    # Statement (s)
    if condition :
        break
    # Statements (s)
    # loop end.
```

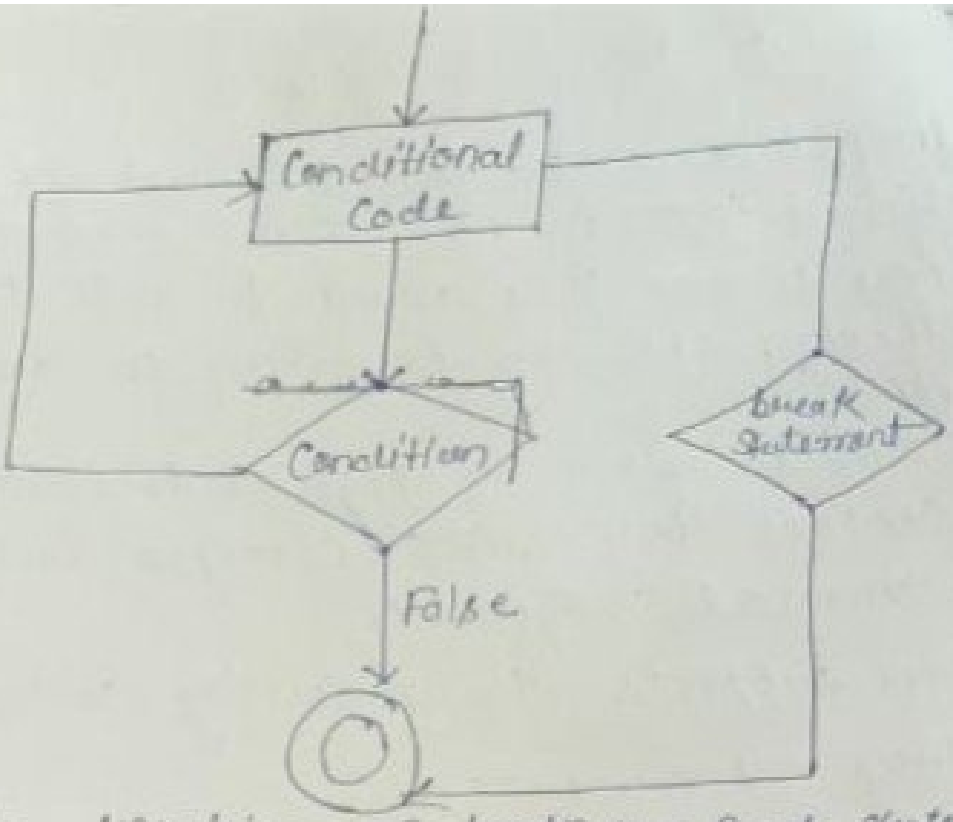


Fig:- working of ~~Loop~~ Python Break Statement

```
# Python program to demonstrate
# break statement with nested
#   for loop
# Use range()
# First for loop
for i in range (1,5):
    # Second for loop
    for j in range (2,6):
        # break the loop if
        # j is divisible by i
            break
        Print (i, " ", j)
```

Out put :-

```
3   2
4   2
4   3
```

Continue statement in Python

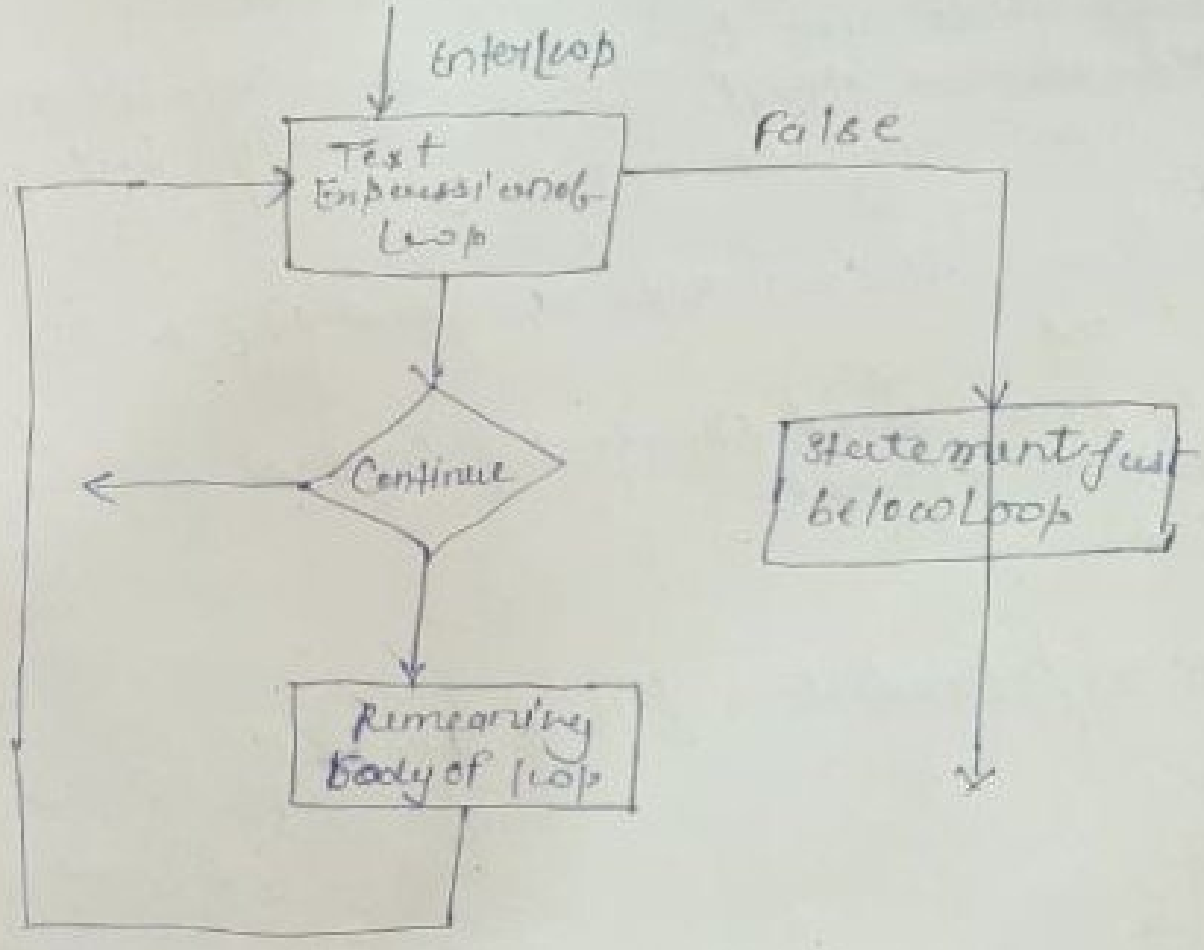
Continue is also a loop control statement just like the break statement.

Continue statement is opposite to that of the break statement, instead of terminating the loop, it forces to execute the next iteration of the loop.

As name suggests the continue statement forces the loop to continue or execute the next iteration.

Syntax:-

```
For / while loop:
    # Statement (s)
    If condition:
        Continue
    # Statement (s)
```



Working of Python Continue statement

```
# Python program to
# demonstrate continue
# statement

# loop from 1 to 10
for i in range(1,11):
    # If i is equals to 6,
    # continue to next iteration
    # without printing
    if i == 6:
        continue
    else:
        # otherwise print the value
        # of i
        print(i, end = " ")
```

output :-

1 2 3 4 5 7 8 9 10

* Pass statement :-

- The name suggests pass statement simply does nothing.
- The pass statement in Python is used when a statement is required syntactically but you do not want any command or code to execute.
- Pass statement can also be used for writing empty loops.
- Pass is also used for empty control statements, functions and classes.

Syntax :-

```
function / condition / loop :
    pass
```